Deadlock Detection

1 Main Idea

Deadlock Detection

Deadlock detection means allowing deadlocks to occur, then checking the system state to find whether a deadlock exists and which processes are involved.

The detection method depends on the resource model:

- If there is **one resource of each type**, use a resource-allocation graph and look for cycles.
- If there are **multiple resources of each type**, use a matrix-based detection algorithm.

2 One Resource of Each Type

Key Rule

When there is only one instance of each resource type, a cycle in the resource-allocation graph means deadlock.

In the graph:

- Process nodes are circles.
- Resource nodes are squares.
- Resource $\rightarrow$ process means the process holds the resource.
- Process $\rightarrow$ resource means the process is waiting for the resource.

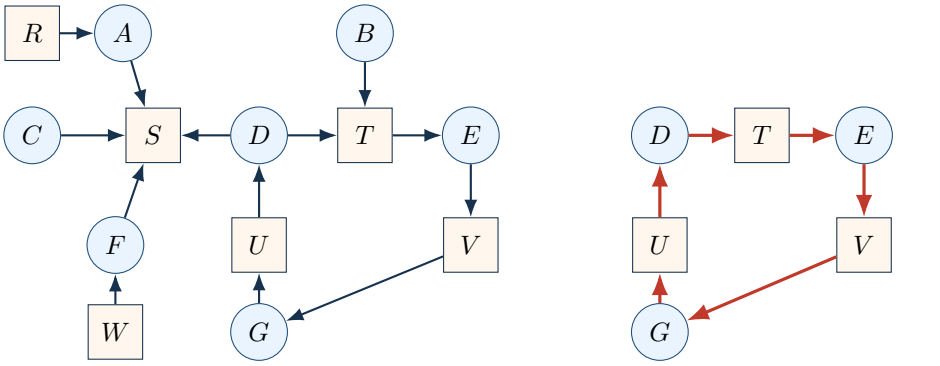
3 Resource Graph Example

System State

There are seven processes A through G, and six resources R through W.

Process	Holds and requests
A	Holds R, wants S.
B	Holds nothing, wants T.
C	Holds nothing, wants S.
D	Holds U, wants S and T.
E	Holds T, wants V.
F	Holds W, wants S.
G	Holds V, wants U.

Resource Graph and Extracted Cycle



(a) full graph **(b) cycle:** D, T, E, V, G, U, D

Conclusion

Processes D, E, and G are deadlocked because they are part of the cycle. Processes A, C, and F are waiting for S, but they are not deadlocked.

4 Cycle Detection Algorithm

For real systems, visual inspection is not enough. The graph can be searched using depth-first search.

Cycle Detection for One Resource of Each Type

```
for each node N in the graph:
    L = empty list
    mark all arcs as uninspected
    current = N

    repeat:
        append current to L
        if current appears twice in L:
            report cycle and stop

        if current has an unmarked outgoing arc:
            mark one such arc
            current = node reached by that arc
        else if current is N:
            stop this search; no cycle from N
        else:
            remove current from L
            current = previous node in L

if all starting nodes are checked:
    report no deadlock
```

How it works

The algorithm starts from each node and performs depth-first search. If the search reaches a node already in the current path L, a cycle has been found.

5 Multiple Resources of Each Type

When there are multiple instances of a resource type, a simple cycle test is not enough. Instead, use vectors and matrices.

Data Structures for Matrix Detection

Existing resources
$E = (E_1, E_2, \ldots, E_m)$

Available resources
$A = (A_1, A_2, \ldots, A_m)$

Current allocation matrix C

$$C = \begin{bmatrix} C_{11} & C_{12} & \cdots & C_{1m} \\ C_{21} & C_{22} & \cdots & C_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ C_{n1} & C_{n2} & \cdots & C_{nm} \end{bmatrix}$$

Row i: resources currently held by process P_i.

Request matrix R

$$R = \begin{bmatrix} R_{11} & R_{12} & \cdots & R_{1m} \\ R_{21} & R_{22} & \cdots & R_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ R_{n1} & R_{n2} & \cdots & R_{nm} \end{bmatrix}$$

Row i: additional resources requested by process P_i.

Symbol	Meaning
E	Existing resource vector. E_j is the total number of resources of class j.
A	Available resource vector. A_j is the number of currently free resources of class j.
C	Current allocation matrix. C_{ij} is how many resources of class j process P_i holds.
R	Request matrix. R_{ij} is how many more resources of class j process P_i wants.

Resource Accounting Invariant

For every resource class j,

$$\sum_{i=1}^{n} C_{ij} + A_j = E_j$$

All resources are either allocated to some process or available.

6 Vector Comparison

For two resource vectors X and Y, write $X \leq Y$ if every component of X is less than or equal to the corresponding component of Y.

$$X \leq Y \iff X_j \leq Y_j \text{ for every } 1 \leq j \leq m$$

This means process P_i's request can be satisfied if row $R_i \leq A$.

7 Matrix Detection Algorithm

Detection with Multiple Resource Instances

```
mark every process as unmarked

repeat:
    find an unmarked process Pi such that Ri <= A

    if such a process exists:
        A = A + Ci
        mark Pi
    else:
        stop

all unmarked processes are deadlocked
```

Meaning

If $R_i \leq A$, process P_i can finish using currently available resources. After it finishes, it releases its current allocation C_i, so A increases.

8 Worked Matrix Example

Setup

There are three processes and four resource classes: tape drives, plotters, scanners, and Blu-ray drives.

$$E = (4, 2, 3, 1), \qquad A = (2, 1, 0, 0)$$

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}, \qquad R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

Step 1: Find a process that can finish

$$R_1 = (2, 0, 0, 1) \not\leq A = (2, 1, 0, 0)$$

Process 1 cannot finish because it needs one Blu-ray drive, but none is available.

$$R_2 = (1, 0, 1, 0) \not\leq A = (2, 1, 0, 0)$$

Process 2 cannot finish because it needs one scanner, but none is available.

$$R_3 = (2, 1, 0, 0) \leq A = (2, 1, 0, 0)$$

Process 3 can finish. Add its allocation $C_3 = (0, 1, 2, 0)$ to A:

$$A = (2, 1, 0, 0) + (0, 1, 2, 0) = (2, 2, 2, 0)$$

Step 2: Process 2 can finish

$$R_2 = (1, 0, 1, 0) \leq A = (2, 2, 2, 0)$$

Add $C_2 = (2, 0, 0, 1)$ to A:

$$A = (2, 2, 2, 0) + (2, 0, 0, 1) = (4, 2, 2, 1)$$

Step 3: Process 1 can finish

$$R_1 = (2, 0, 0, 1) \leq A = (4, 2, 2, 1)$$

All processes can be marked, so there is no deadlock.

Answer

The system is not deadlocked. A possible completion order is $P_3 \to P_2 \to P_1$.

9 When to Run Detection

Detection time	Tradeoff
After every resource request	Finds deadlocks early, but uses more CPU time.
Every k minutes	Reduces overhead, but deadlocks may persist longer.
When CPU utilization is low	Useful because many deadlocked processes leave few runnable processes.